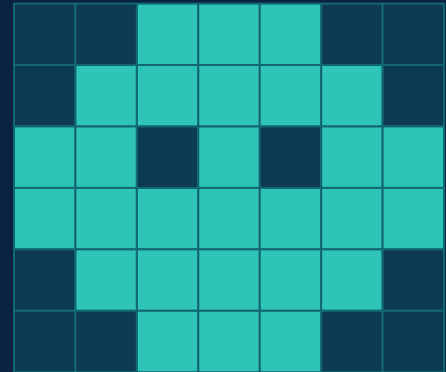


Morphological Image Processing

Topics Covered

Erosion & Dilation • Opening & Closing
Boundary Extraction • Region Filling
Structuring Elements • Morphological Algorithms

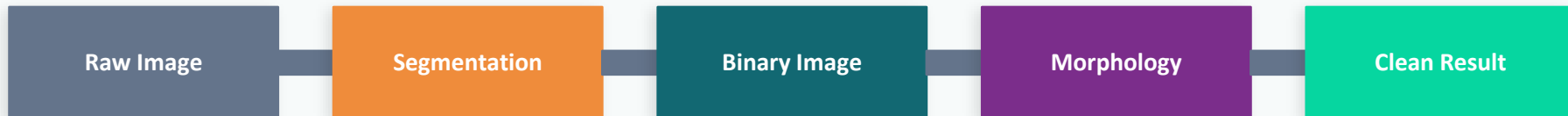


Pixel Object

What is Morphological Image Processing?

Definition

Morphological processing is a set of image processing techniques based on the shape (morphology) of features in an image. It operates on binary (segmented) images to clean, analyze, and extract structure.



Processing Pipeline



Binary Images

Works on images with only 0s (background) and 1s (object pixels) — after segmentation has been performed.



Remove Imperfections

After segmentation, images often have noise, holes, and unwanted pixels. Morphology cleans these up systematically.



Shape Analysis

Unlike filtering (which looks at intensity), morphology focuses on the geometric shape and structure of objects.

Binary Images: 0s and 1s

All morphological operations work on binary images where:

0

Background Pixel

Pixels that do NOT belong to any object. Usually displayed as white or black depending on convention.

1

Object Pixel

Pixels that belong to an object of interest. These are the pixels we analyse and modify.

Important Note

Whether 0 is displayed as black or white depends on the software and convention used. What matters is that 0 = background and 1 = object.

Once you understand this, the colour rendering doesn't matter — all operations work on the numerical values, not the visual colour.

Structuring Elements

A structuring element (SE) is a small binary matrix used as a probe to examine the image. Think of it like a stamp or template.

Key Properties of a Structuring Element:

- Can be any size and any shape
- Has an origin (usually the centre pixel)
- Placed over each pixel of the image in turn
- The result depends on how well it 'fits' or 'hits' the image

Two Key Concepts: HIT and FIT

FIT

ALL '1' pixels in the SE overlap with '1' pixels in the image. The SE fits perfectly inside the object.

Used in: Erosion

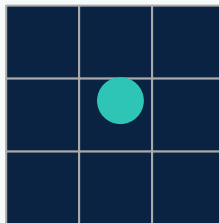
HIT

AT LEAST ONE '1' pixel in the SE overlaps with a '1' pixel in the image. Even partial overlap counts.

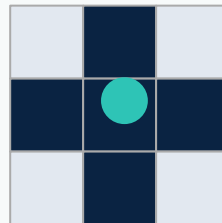
Used in: Dilation

Common Shapes:

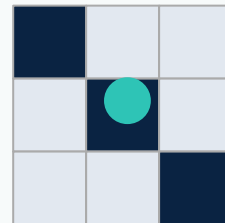
3×3 Square



Cross



Diagonal



● = Origin pixel

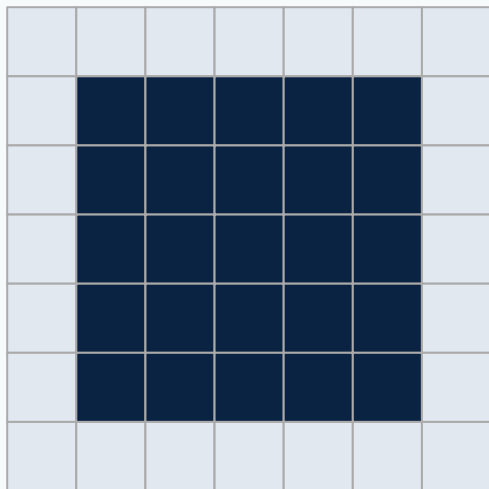
Erosion — Shrinking Objects

$f \ominus s$: Keep a pixel only if the SE completely fits inside the object

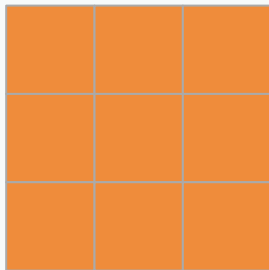
Rule: A pixel in the output is set to 1 ONLY IF the structuring element fits entirely within the object when placed over that pixel.

💡 Intuition: Erosion eats away at the edges of an object. The more erosion, the thinner the object becomes.

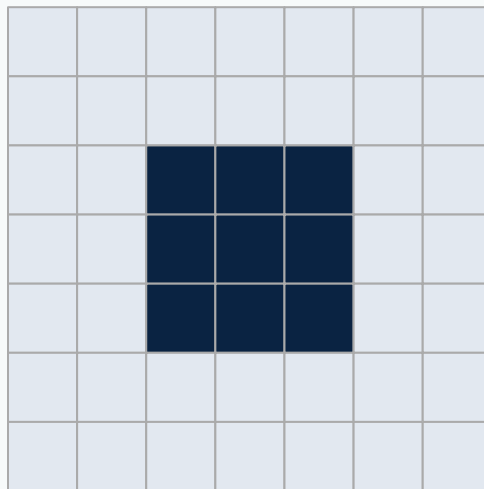
Original Binary Image



3x3 Square SE



Eroded Result



- ☒ Splits joined objects
- ☒ Removes small noise
- ☒ Strips thin protrusions
- ☐ ⚠ Shrinks all objects

Erosion — Step-by-Step Walkthrough

How to manually apply erosion

Place the SE origin over each pixel. If all SE '1's align with image '1's → output is 1. Otherwise → output is 0.

Step 1: Original Image (5×5)

0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

Step 2: 3×3 SE

1	1	1
1	1	1
1	1	1

Step 3: Test centre pixel (2,2)

✓ SE fits perfectly
→ Output pixel = 1

Step 4: Eroded Output (only centre survives!)

0	0	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0

Why only 1 pixel survives?

The 3×3 SE only fits when ALL 9 positions align with 1s. In this 3×3 object (surrounded by 0s), only the centre pixel (2,2) satisfies this — every surrounding position would expose at least one 0.

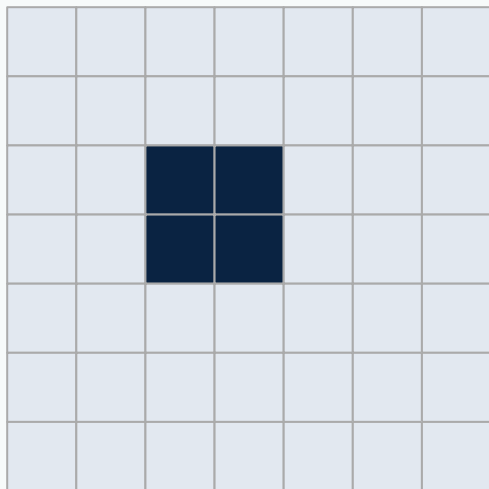
Dilation — Expanding Objects

$f \oplus s$: Set a pixel to 1 if the SE hits any object pixel

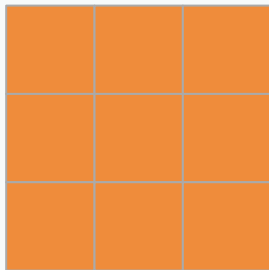
Rule: A pixel in the output is set to 1 IF the structuring element, placed over that pixel, overlaps with at least one '1' pixel in the image.

💡 Intuition: Dilation grows the object outward. Think of it as adding a thick border around every object pixel.

Original Binary Image

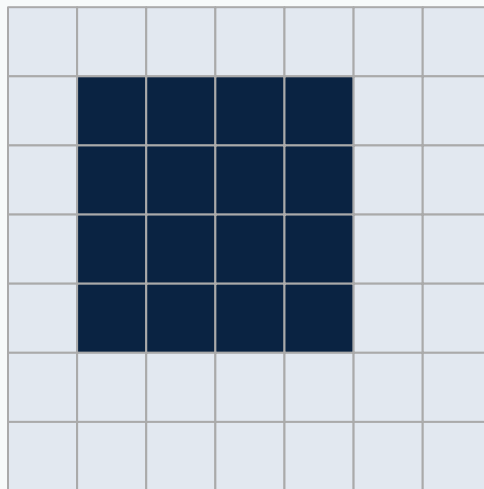


3x3 Square SE



Dilation

Dilated Result



- ☒ Repairs broken objects
- ☒ Fills small holes
- ☒ Bridges nearby gaps
- ☐ ⚠ Enlarges all objects

Erosion vs Dilation — Side-by-Side Comparison

Understanding the opposite effects

Property	Erosion ($\ominus$)	Dilation ($\oplus$)
Rule	SE must FIT (all 1s match)	SE must HIT (any 1 matches)
Effect on object	Shrinks / reduces object	Grows / expands object
Removes	Small objects, thin lines	Small holes, gaps
Main use	Separate joined objects	Connect broken objects
Risk	Can erase small objects	Can merge close objects

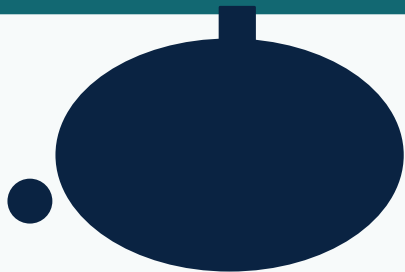
Opening — Removing Small Objects & Protrusions

$f \circ s = (f \ominus s) \oplus s$ (Erosion followed by Dilation)

Opening = first erode, then dilate with the same SE. This removes small objects and thin connections while largely restoring the original size of larger objects.

$$f \circ s = (f \ominus s) \oplus s$$

Original Image



Object with thin protrusions & noise

After Erosion



Thin parts removed, larger objects shrunk

After Dilation
(= Opening)



Larger objects restored to original size

What Opening preserves vs removes:

Keeps: Objects larger than the SE. Removes: Objects smaller than the SE, thin protrusions, noise points.

Why dilation after erosion? The erosion removes small things; the dilation re-grows the surviving larger objects back to ~original size.

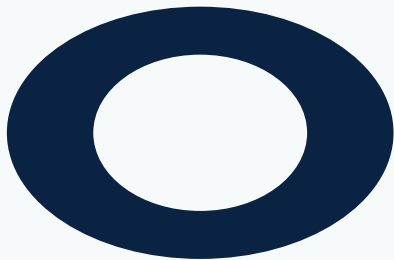
Closing — Filling Holes & Connecting Gaps

$f \bullet s = (f \oplus s) \ominus s$ (Dilation followed by Erosion)

Closing = first dilate, then erode with the same SE. This fills small holes and bridges narrow gaps while largely preserving the overall shape.

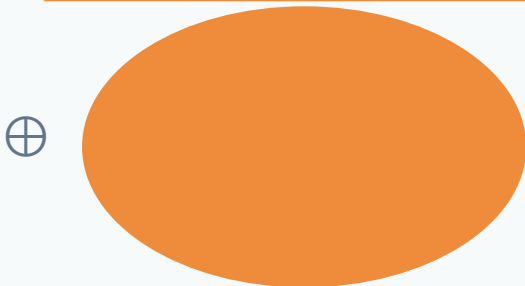
$$f \bullet s = (f \oplus s) \ominus s$$

Original Image



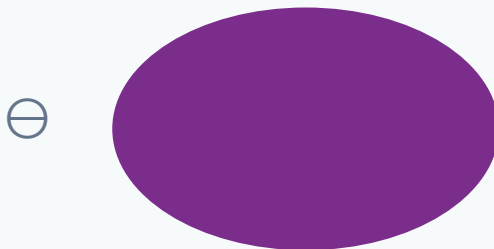
*Object with holes
and narrow gaps*

After Dilation



*Holes filled, gaps
bridged (object enlarged)*

After Erosion
(= Closing)



*Object returns to ~original
size but with holes filled*

Opening ($\ominus$ then $\oplus$)

→ Removes small bright regions & thin connections in foreground

Closing ($\oplus$ then $\ominus$)

→ Fills small dark holes & bridges narrow gaps in foreground

Opening & Closing — Summary & Applications

Opening $f \circ s$

Operation: Erosion $\rightarrow$ Dilation

Applications:

- Remove salt noise (white dots on dark background)
- Separate touching objects
- Remove thin protrusions
- Smooth object boundaries

 Like sanding rough edges off an object — removes bumps but keeps the core shape.

Closing $f \bullet s$

Operation: Dilation $\rightarrow$ Erosion

Applications:

- Fill pepper noise (dark holes in bright objects)
- Connect nearby broken objects
- Fill small interior holes
- Smooth object boundaries from inside

 Like filling in cracks or gaps — plugs holes but keeps the overall shape.

 Real-world use: In document scanning, opening removes specks of ink dust; closing fills broken characters. In medical imaging, closing fills gaps in cell boundaries.

Boundary Extraction

$\beta(A) = A - (A \ominus B)$: Find the outline of an object

Boundary extraction finds the contour (edge/outline) of binary objects. It's one of the most fundamental morphological algorithms.

$$\beta(A) = A - (A \ominus B)$$



Start with
binary image A



Erode A with
a small SE B

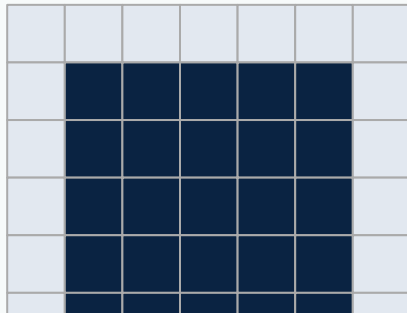


Subtract eroded
from original

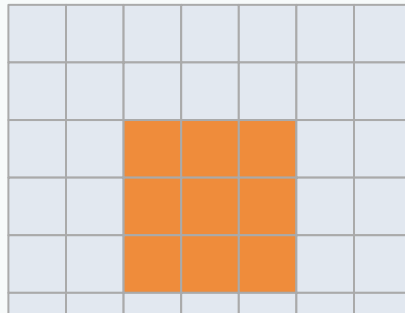


Result: only
boundary pixels

A (Original)



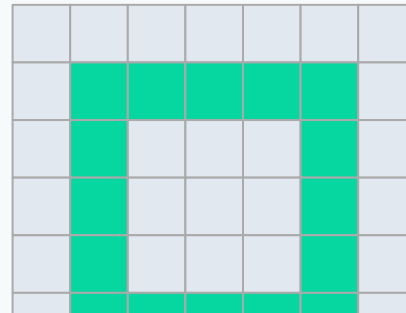
$A \ominus B$



-

=

$\beta(A)$ Boundary



Only the
border ring
survives!

Thicker SE
= thicker
boundary

Region Filling

Region filling is a morphological algorithm that, given a single seed pixel located inside a closed boundary, fills the entire enclosed region with object pixels (1s).

Prerequisites

Before applying region filling, we need:

- A binary image where the boundary is made of 1s and the background/interior is 0s
- A seed point — one known pixel that lies inside the boundary (given manually or detected automatically)
- A structuring element B — typically a 3×3 cross or square

Formula:

$$X_k = (X_{k-1} \oplus B) \cap A^c \quad (\text{repeated until } X_k = X_{k-1})$$

Region Filling

Let's break down every symbol:

X_0

Seed point — the starting pixel inside the boundary (given by the user)

B

Structuring element (usually a simple cross or 3×3 square)

A^c

Complement of the boundary image A (i.e., NOT A — background pixels)

$\cap$

Intersection — keeps only pixels that are also background (stops at boundary)

X_k

Set of filled pixels at iteration k — grows each step until boundary is hit

Iteration Steps (Example — Circular Boundary):



Region Filling - Step by Step

Step 0 — Set Up:

- Let A = the binary image containing the closed boundary (boundary pixels = 1)
- Compute A^c (the complement) — flip all pixels: boundary pixels become 0, everything else becomes 1
- Choose your seed point — one pixel inside the boundary — call this X_0
- So $X_0 = \{\text{one single pixel with value 1, located inside the boundary}\}$

Step 1 — First Iteration ($k = 1$)

Apply the formula:

$$X_k = (X_{k-1} \oplus B) \cap A^c \quad (\text{repeated until } X_k = X_{k-1})$$

What happens:

1. Dilate X_0 with $B \rightarrow$ the single seed pixel expands outward, creating a small cluster of 1s around the seed
2. Intersect with $A^c \rightarrow$ any newly dilated pixels that landed on the boundary (which are 0 in A^c) are removed
3. Result: X_1 is slightly larger than X_0 , but still contained inside the boundary

Region Filling - Step by Step

Step 2 — Second Iteration (k = 2)

$$X_2 = (X_1 \oplus B) \cap A^c \quad (\text{repeated until } X_k = X_{k-1})$$

What happens:

1. Dilate $X_1 \rightarrow$ the small cluster from Step 1 expands further outward in all directions
2. Intersect with $A^c \rightarrow$ again, any pixels that touch the boundary wall are cut off
3. Result: X_2 is larger than X_1 , spreading further toward the boundary

Region Filling - Step by Step

Step 3, 4, 5... — Repeat

Keep repeating the same formula, incrementing k each time. With every iteration:

- The filled region grows outward like ripples in water
- The boundary acts as a wall — the intersection with A^c prevents the fill from leaking through or past the boundary
- The fill is always contained inside the closed region

Final Step — Stopping Condition

Stop when:

$$X_k = X_{k-1}$$

This means the dilation produced no new pixels — the filled region has expanded as far as it can go and is pressing against the boundary on all sides. The region is now completely filled.

Region Filling - Step by Step

Finally, combine the filled interior with the original boundary:

$$\mathbf{Result} = \mathbf{X_k} \cup \mathbf{A}$$

This gives the complete filled shape — interior pixels (from X_k) plus the boundary pixels (from A) — a fully solid object.

Region Filling - Example

Concrete Numerical Example

Suppose you have this 7×7 binary image (1 = boundary, 0 = inside/background):

```
0 0 0 0 0 0 0
0 1 1 1 1 1 0
0 1 0 0 0 1 0
0 1 0 0 0 1 0
0 1 0 0 0 1 0
0 1 1 1 1 1 0
0 0 0 0 0 0 0
```

Seed point X_0 = pixel at position (3, 3) — the centre of the hollow square.

A^c (complement of A): flip every pixel — boundary becomes 0, interior and outer background become 1:

```
1 1 1 1 1 1 1
1 0 0 0 0 0 1
1 0 1 1 1 0 1
1 0 1 1 1 0 1
1 0 1 1 1 0 1
1 0 0 0 0 0 1
1 1 1 1 1 1 1
```

Region Filling - Example

Iteration 1: Dilate the seed $\rightarrow 3 \times 3$ region centred at (3,3) $\rightarrow$ intersect with $A^c \rightarrow$ only the 3 interior pixels survive. X_1 grows slightly.

Iteration 2: Dilate $X_1 \rightarrow$ expands to more interior pixels $\rightarrow$ intersect stops it at the boundary wall.

Iteration 3: X_3 fills all 9 interior pixels — the entire hollow interior is now 1.

Stopping: $X_3 = X_2$ (no change) $\rightarrow$ stop.

Union with A: interior 1s + boundary 1s = solid filled square.

Why the Intersection with A^c is Critical

Without the $\cap A^c$ step, the dilation would **leak through the boundary** and flood the entire image. The complement A^c acts as a **mask** — it lets the fill grow freely inside but blocks it the moment it touches a boundary pixel. This  is a key insight of the algorithm.

Summary — Morphological Image Processing

0 / 1	Binary Images Morphology works on segmented images — 0 = background, 1 = object. Colour display doesn't matter.	$\ominus \rightarrow \oplus$	Opening Erosion then dilation. Removes small bright objects & protrusions, preserves larger shapes.
SE	Structuring Element Small binary probe. Operations depend on how SE relates to image pixels (HIT or FIT).	$\oplus \rightarrow \ominus$	Closing Dilation then erosion. Fills holes & narrow gaps, preserves overall shape.
$\ominus$	Erosion Shrinks objects. Keeps pixel only if SE fully fits. Removes noise, splits touching objects.	$\beta(A)$	Boundary Extraction $\beta(A) = A - (A \ominus B)$. Subtracts eroded image from original to keep only the border pixels.
$\oplus$	Dilation Grows objects. Sets pixel if SE hits any object pixel. Repairs breaks, fills gaps.	X_k	Region Filling Iterative dilation from seed point, masked by boundary complement. Fills enclosed regions.